

Mathematical Background and Implementation Guide

n2_theory_index.py

FORM and LiE computation of four-dimensional $\mathcal{N} = 2$ indices

Simple and semisimple product gauge groups

Revised 1 September 2026

Purpose. The canonical implementation computes the flavor-unrefined right superconformal index of a Lagrangian four-dimensional $\mathcal{N} = 2$ gauge theory. FORM performs exact truncated series algebra. LiE decomposes Adams-operated characters and tensor products. Product gauge groups are handled by projecting to the trivial representation independently in every simple factor. SageMath remains only in the shared input/representation validation layer and in the returned exact polynomial ring; it no longer performs the index expansion or Haar projection.

Contents

1	Scope and program architecture	2
2	Index convention and single-letter functions	2
3	Matter characters for simple and product groups	3
4	Plethystic exponential and exact cutoffs	3
5	How FORM represents and expands the index	3
6	Adams operations and LiE decomposition	5
7	Gauge-singlet projection for product groups	6
8	The file-backed character cache	6
9	Python functions and their mathematical roles	7
10	JSON examples and command-line use	7
11	Validation and representative performance	8
12	Interpretation and limitations	8
	References	9

1 Scope and program architecture

The supported gauge algebra is a finite semisimple algebra

$$\mathfrak{g} = \bigoplus_{a=1}^s \mathfrak{g}_a, \quad G = \prod_{a=1}^s G_a, \quad (1)$$

where every $\mathfrak{g}_a$ has Cartan type $A_r, B_r, C_r, D_r, E_6, E_7, E_8, F_4$, or G_2 . The compact group convention is the simply connected one used by the shared Lie-algebra module; in particular C_r is written $\mathrm{Sp}(r)$. Abelian gauge factors are outside the current input model.

The calculation has four layers:

1. `check_input_data` validates the existing simple- or product-group JSON convention and resolves names to Dynkin labels.
2. `_build_form_program` constructs the truncated plethystic exponential with formal character symbols.
3. `CharacterDecompositionCache` asks LiE for the necessary virtual character decompositions and saves them as structured JSON.
4. `_project_terms` extracts the gauge singlet factor by factor and `_to_sage_polynomial` packages the exact answer.

This division is important: FORM never needs to know a root system, while LiE never needs to manipulate the fugacities t, y, u .

2 Index convention and single-letter functions

Choose the supercharge defining the right index. With the conventions of Gadde, Pomoni, and Rastelli, the BPS condition and trace are [1]

$$\delta_R = \Delta - 2\bar{j} - 2R + r = 0, \quad (2)$$

$$\mathcal{I}(t, y, v) = \mathrm{Tr}_{\delta_R=0} (-1)^F t^{2(\Delta+\bar{j})} y^{2j} v^{-r-R}. \quad (3)$$

The code substitutes

$$v = u^2, \quad (4)$$

so every computational power of u is integral.

The vector and half-hypermultiplet single-letter indices are [1]

$$f_V(t, y, u) = \frac{t^2 u^2 - t^4 u^{-2} - t^3(y + y^{-1}) + 2t^6}{(1 - t^3 y)(1 - t^3 y^{-1})}, \quad (5)$$

$$f_H(t, y, u) = \frac{t^2 u^{-1} - t^4 u}{(1 - t^3 y)(1 - t^3 y^{-1})}. \quad (6)$$

The denominator counts the two compatible derivatives:

$$\frac{1}{(1 - t^3 y)(1 - t^3 y^{-1})} = \sum_{p, q \geq 0} t^{3(p+q)} y^{p-q}. \quad (7)$$

FORM uses a finite rectangular sum with $0 \leq p, q \leq \lfloor D/3 \rfloor$. Terms whose combined t degree exceeds D are removed automatically.

3 Matter characters for simple and product groups

For a product group, an irreducible external tensor product is

$$\mathbf{R}_h = \boxtimes_{a=1}^s R_{h,a}, \quad \chi_{\mathbf{R}_h}(g_1, \dots, g_s) = \prod_{a=1}^s \chi_{R_{h,a}}(g_a). \quad (8)$$

An omitted representation in the JSON input is the singlet, whose character is one.

The adjoint representation of the product algebra is a direct sum,

$$\mathfrak{ad}_{\mathfrak{g}} = \bigoplus_{a=1}^s (\mathbf{1}, \dots, \mathfrak{ad}_{\mathfrak{g}_a}, \dots, \mathbf{1}), \quad \chi_{\mathfrak{ad}_{\mathfrak{g}}} = \sum_{a=1}^s \chi_{\mathfrak{ad}_a}. \quad (9)$$

This explains why vector-multiplet characters are added, whereas a bifundamental hypermultiplet character is multiplied across its gauge factors.

For multiplicity N_h , the character measured in half-hyper units is

$$\hat{\chi}_h = \begin{cases} N_h \chi_{\mathbf{R}_h}, & \text{half hyper,} \\ N_h (\chi_{\mathbf{R}_h} + \chi_{\overline{\mathbf{R}}_h}), & \text{full hyper.} \end{cases} \quad (10)$$

Conjugation is simultaneous and factorwise:

$$\overline{\mathbf{R}}_h = \boxtimes_{a=1}^s \overline{R}_{h,a}. \quad (11)$$

Thus a full hyper in $R_1 \boxtimes R_2$ contains $R_1 \boxtimes R_2$ and $\overline{R}_1 \boxtimes \overline{R}_2$; it does not contain the two mixed tensor products unless they are separate matter entries.

4 Plethystic exponential and exact cutoffs

With flavor fugacities unrefined, define the representation-valued letter index

$$\mathcal{L}(t, y, u; g) = f_V(t, y, u) \sum_{a=1}^s \chi_{\mathfrak{ad}_a}(g_a) + f_H(t, y, u) \sum_h \hat{\chi}_h(g). \quad (12)$$

The free-field index before gauge projection is the plethystic exponential [1, 2]

$$\mathcal{Z}(t, y, u; g) = \text{PE}[\mathcal{L}] = \exp E(t, y, u; g), \quad (13)$$

$$E(t, y, u; g) = \sum_{j=1}^{\infty} \frac{1}{j} \mathcal{L}(t^j, y^j, u^j; g^j). \quad (14)$$

Since every letter begins at t^2 , a result through t^D needs only

$$1 \leq j \leq M = \left\lfloor \frac{D}{2} \right\rfloor. \quad (15)$$

The same lower bound implies that only $E^k/k!$ with $k \leq M$ can contribute. Consequently both the Adams sum and the ordinary exponential have finite, exact cutoffs at the requested order.

5 How FORM represents and expands the index

5.1 Formal character symbols

Each distinct pair “gauge-factor position, Dynkin labels” receives a commuting FORM function $C_p(j)$. Mathematically it denotes the j -th Adams operation

$$C_p(j) = \psi^j(\chi_{R_p}), \quad (\psi^j \chi)(g) = \chi(g^j). \quad (16)$$

For a bifundamental, FORM therefore sees $C_p(j)C_q(j)$. It expands only the coefficients and products of these formal objects; their Lie-theoretic decomposition is postponed until after the series is complete.

At Adams degree j , `_build_form_program` constructs

$$\mathcal{L}_j = f_V(t^j, y^j, u^j) \sum_{p \in \mathcal{V}} C_p(j) + f_H(t^j, y^j, u^j) \sum_m N_m \prod_{p \in m} C_p(j), \quad (17)$$

where $\mathcal{V}$ contains the adjoint character of every gauge factor and the matter monomials already include the full-hyper conjugate term from Eq. (10).

5.2 The generated FORM program in plain language

The central generated block has the following form:

Listing 1: Schematic FORM program generated by Python.

```
S m,n,y,idx1,idx2,j,z,u,t(:D);
CF d,C0,C1,...;
PolyRatFun d;
Function Kvec,Khyp;

L J=sum_(idx1,0,D/3,m^idx1)
    *sum_(idx2,0,D/3,n^idx2);
id m=t^3*y;
id n=t^3/y;
.sort

L itotal=sum_(j,1,D/2,(vector terms + hyper terms)/j);
id Kvec(t?,y?,u?)=J*(t^2*u^2-t^4/u^2-t^3*y-t^3/y+2*t^6);
id Khyp(t?,y?,u?)=J*(t^2/u-t^4*u);
.sort
```

FORM construct	Meaning in this calculation
S ... t(:D);	Declares commuting symbols. The maximum-power declaration automatically discards $t^{D+1}, t^{D+2}, \dots$ during term normalization [4, Sec. 7.143].
CF d,C0,...;	Declares commuting functions. The $C_p(j)$ objects are formal Adams-operated characters.
PolyRatFun d;	Makes <code>d(numerator,denominator)</code> the exact rational coefficient container, so equal monomials combine without floating point arithmetic [4, Sec. 7.114].
L name=...;	Creates a local FORM expression. <code>J</code> , <code>itotal</code> , <code>I</code> , and <code>result</code> are successive exact expressions.
sum_(j,a,b,X)	Generates the finite sum $\sum_{j=a}^b X$; the last argument is the summand [4, Sec. 8.67].
id lhs=rhs;	Applies an algebraic substitution. A question mark denotes a wildcard, so <code>Kvec(t?,y?,u?)</code> matches arguments such as <code>Kvec(t^j,y^j,u^j)</code> [4, Secs. 7.70–7.71].
.sort	Ends a FORM module, expands products, combines identical terms, and enforces the t cutoff before the next module [4, Sec. 2.3].

5.3 Why the auxiliary symbol z gives the exponential

Let $E = \text{itotal}$ and $M = \lfloor D/2 \rfloor$. The code begins with $I = zE$. For $k = 2, \dots, M$ it applies

$$z \longmapsto 1 + \frac{zE}{k}. \quad (18)$$

After the k -th step, induction gives

$$I_k = \sum_{r=1}^{k-1} \frac{E^r}{r!} + z \frac{E^k}{k!}. \quad (19)$$

The final substitution $z \mapsto 1$, followed by adding the vacuum term, yields

$$1 + I_M = \sum_{r=0}^M \frac{E^r}{r!} = \exp(E) + O(t^{D+1}). \quad (20)$$

This is an ordinary exponential of a representation-valued series; Bose and Fermi statistics have already entered through the signed single-letter functions.

6 Adams operations and LiE decomposition

6.1 The representation-ring calculation

For each irrep R and exponent vector $\mathbf{n} = (n_1, n_2, \dots)$, the cache stores the virtual decomposition

$$\mathcal{D}_R(\mathbf{n}) = \text{Decomp} \left[\bigotimes_{j \geq 1} (\psi^j R)^{\otimes n_j} \right]. \quad (21)$$

An Adams operation scales every weight by j and can produce a virtual representation with negative coefficients [3, Secs. 3.4, 4.5]. For the A_1 fundamental,

$$\psi^2(\mathbf{2}) = \mathbf{3} - \mathbf{1}, \quad (22)$$

which LiE prints as a signed decomposition polynomial.

6.2 The LiE commands in plain language

LiE expression	Meaning
<code>Adams(2, [1], A1)</code>	Decomposes ψ^2 of the A_1 irrep with Dynkin label [1]. The result is $-1X[0] + 1X[2]$ [3, Sec. 4.5].
<code>tensor([1], [1], A1)</code>	Decomposes $V_{[1]} \otimes V_{[1]}$ into irreducibles. LiE returns $1X[0] + 1X[2]$ [3, Sec. 4.5].
<code>tensor(p, q, A1)</code>	Tensors two decomposition polynomials p and q . The cache nests this call when more than two factors occur.
<code>tensor(p, q, [0], A1)</code>	Requests only the coefficient of the trivial irrep [0]. The singlet projector uses this targeted form instead of retaining the full result.
<code>maxnodes N / maxobjects N</code>	Raises LiE's program-node or object-table limit [3, Sec. 2.3]. The Python code retries with these settings only after a failed ordinary run, because eager large allocation has substantial startup cost.

A decomposition polynomial has the form

$$p = \sum_{\lambda \in P_+} m_\lambda X[\lambda_1, \dots, \lambda_r], \quad m_\lambda \in \mathbb{Z}. \quad (23)$$

The parser preserves the signed integer m_λ . It also removes LiE's optional "New tree space with maximum number of nodes" notification by recognizing the message prefix, rather than by deleting a fixed number of output characters.

7 Gauge-singlet projection for product groups

The gauge index is the Haar projection

$$\mathcal{I}(t, y, u) = \int_G d\mu_G(g) \mathcal{Z}(t, y, u; g). \quad (24)$$

Equivalently, it is the multiplicity of the trivial representation in every coefficient.

After FORM expansion, a typical formal-character monomial is

$$\mathcal{C} = \prod_p \prod_{j \geq 1} C_p(j)^{n_{p,j}}. \quad (25)$$

If $a(p)$ is the gauge factor associated with character p , define the virtual G_a representation

$$X_a(\mathcal{C}) = \bigotimes_{p: a(p)=a} \bigotimes_{j \geq 1} (\psi^j R_p)^{\otimes n_{p,j}}. \quad (26)$$

Because both the group and normalized Haar measure factorize,

$$d\mu_G = \prod_{a=1}^s d\mu_{G_a}, \quad (27)$$

$$\text{mult}_{\mathbf{1}_G}(\mathcal{C}) = \prod_{a=1}^s \text{mult}_{\mathbf{1}_a}(X_a(\mathcal{C})). \quad (28)$$

This is the precise rule implemented by `_project_terms`. It partitions a FORM term by gauge-factor position, batches all distinct LiE questions for one factor, then multiplies the returned singlet multiplicities. Empty character products have singlet multiplicity one.

8 The file-backed character cache

For the exponent vector in Eq. (21), define its total Adams order

$$q(\mathbf{n}) = \sum_{j \geq 1} j n_j. \quad (29)$$

The canonical path is

```
char_decomposition_cache_data/<Cartan>/
<Cartan>_dynkin_<labels>_adams_order_<q>.json
```

For example,

```
A2/A2_dynkin_1-0_adams_order_4.json
G2/G2_dynkin_0-1_adams_order_3.json
```

There is no combined filename for a product group. Each simple factor is decomposed separately, so identical factors and representations reuse the same files. A cache file contains metadata plus a list of entries of the form

```
{
  "adams_powers": [n1, n2, ...],
  "terms": [
    {"coefficient": m, "dynkin_labels": [a1, a2, ...]}
  ]
}
```

Writes use a temporary file followed by an atomic replacement. A cold-cache run asks LiE to generate missing entries; a warm-cache run reads them from JSON. FORM still performs the fugacity expansion on every run.

9 Python functions and their mathematical roles

Function or class	Mathematical/computational role
<code>_parse_input</code>	Resolves the ordered factors in Eq. (1) and validated hypermultiplet data.
<code>_matter_character_multiplicities</code>	Constructs Eq. (10), omits trivial factor characters, and applies conjugation as in Eq. (11).
<code>_character_basis</code>	Assigns each pair (a, R) one FORM function C_p and records vector and matter character monomials.
<code>_build_form_program</code>	Implements Eqs. (7), (17), (14), and (20).
<code>_run_form</code>	Runs FORM in an isolated temporary directory and requires a successful, clean process result.
<code>_parse_form_output</code>	Converts each exact FORM monomial to a <code>FormTerm</code> ; rational coefficients become Python <code>Fraction</code> objects.
<code>CharacterDecompositionCache. get_decomposition CharacterDecompositionCache. singlet_multiplicities</code>	Retrieves or constructs Eq. (21). Uses targeted LiE tensor products to compute mult_{1_a} in Eq. (28).
<code>_project_terms</code>	Multiplies factorwise singlet multiplicities and collects equal powers of t, y, u .
<code>_to_sage_polynomial</code>	Returns an exact element of $\mathbb{Q}[y^{\pm 1}, u^{\pm 1}][t]$; it performs no representation calculation.
<code>calculate_index</code>	Coordinates the complete public in-memory calculation.
<code>calculate_index_from_file</code>	Loads JSON and delegates to <code>calculate_index</code> .

10 JSON examples and command-line use

A simple-factor theory retains the existing form:

```
{
  "algebra": "A2",
  "hypermultiplets": [
    {"representation": "fundamental", "number": 6, "kind": "full"}
  ]
}
```

A product theory names every gauge factor. Missing entries in `representations` mean singlets:

```
{
  "gauge_groups": [
    {"id": "left", "algebra": "A1"},
    {"id": "right", "algebra": "A1"}
  ],
  "hypermultiplets": [{
    "representations": {
      "left": "fundamental", "right": "fundamental"
    },
    "number": 2, "kind": "full"
  }]
}
```

With the Sage environment active and `form` and `lie` on PATH:

```
python -m index.n2_theory_index theory.json --order 10
```

The default cache is `index/char_decomposition_cache_data`; alternate paths are accepted.

11 Validation and representative performance

The former sparse Sage weight-lattice implementation was retained long enough to serve as an independent reference before replacement. Exact comparisons gave:

Theory	Order	Check
$SU(2)$ with four full fundamentals	t^{18}	exact agreement
$SU(3)$ with six full fundamentals	t^{18}	exact agreement
$SU(4)$ with eight full fundamentals	t^{18}	exact agreement
$SU(2) \times SU(2)$ bifundamental theory	t^{10}	independent exact agreement
Decoupled product theories	t^6	factorization identity
$SU(2)^3$ trifundamental half hyper	t^6	factor-order invariance

For conformal $SU(3)$ SQCD, the first nonzero terms are

$$\mathcal{I} = 1 + t^4 \left(\frac{36}{u^2} + u^4 \right) - t^5 u^2 (y + y^{-1}) + t^6 \left(\frac{40}{u^3} - 36 + u^6 \right) + O(t^7). \quad (30)$$

Representative product-group timings through t^{10} were:

Theory	Cold cache	Warm cache
$A_1 \times A_1$ bifundamental	1.030 s	0.042 s
$A_1 \times G_2$ mixed matter	0.274 s	0.127 s

Timings are machine-dependent. They illustrate that the persistent decomposition cache removes repeated LiE work but not FORM's per-run series expansion.

The complete project test run after replacement passed 55 tests; two live MySQL integration tests were skipped because no test database was requested.

12 Interpretation and limitations

- The result is a signed protected index, not a complete operator spectrum.
- Flavor fugacities are unrefined. Matter multiplicities are integer coefficients.
- The code supports semisimple products of the listed finite simple algebras, but not $U(1)$ gauge factors.
- The Lie algebra does not determine the global form of the gauge group, discrete theta data, or the spectrum of line operators.
- Input validation checks representation syntax and physical admissibility of a half hyper. The index routine does not require the beta functions or global gauge anomalies to vanish; those are separate checker outputs.
- High orders still produce many FORM monomials and many LiE tensor questions. The cache reduces repeated decomposition work but cannot remove the intrinsic combinatorial growth of the plethystic exponential.

References

- [1] A. Gadde, E. Pomoni, and L. Rastelli, “The Veneziano Limit of $\mathcal{N} = 2$ Superconformal QCD: Towards the String Dual of $\mathcal{N} = 2$ $SU(N_c)$ SYM with $N_f = 2N_c$,” *JHEP* 03 (2010) 032, [arXiv:0912.4918](#).
- [2] S. Benvenuti, B. Feng, A. Hanany, and Y.-H. He, “Counting BPS Operators in Gauge Theories: Quivers, Syzygies and Plethystics,” *JHEP* 11 (2007) 050, [arXiv:hep-th/0608050](#).
- [3] M. A. A. van Leeuwen, A. M. Cohen, and B. Lisser, *LiE, A Package for Lie Group Computations*, manual, 1992. In particular, Secs. 3.4 and 4.5 describe Adams operations and tensor-product decomposition polynomials.
- [4] J. Kuipers, J. A. M. Vermaseren, and collaborators, *FORM 5.0.0 Reference Manual*, 2026.
- [5] J. Kuipers, T. Ueda, J. A. M. Vermaseren, and J. Vollinga, “FORM version 4.0,” *Comput. Phys. Commun.* 184 (2013) 1453–1467, [arXiv:1203.6543](#).